

# **Profiling and Analyzing the Performance of Irregular Applications on High-Performance Processors**

*Responsible: Ajeya Naithani ([a.naithani@tue.nl](mailto:a.naithani@tue.nl))*

## **Introduction and Context**

Graphs are widely used to represent relationships in social networks, road networks, financial transactions, biological systems, recommendation platforms, and communication networks. Graph analytics extracts useful information from these structures by applying algorithms such as Breadth-First Search (BFS), Single-Source Shortest Path (SSSP), and PageRank, etc. As modern graphs may contain millions or billions of vertices and edges, their processing has considerable commercial and scientific importance.

Graph applications differ from conventional compute-intensive workloads because they exhibit irregular memory accesses, data-dependent control flow, large working sets, and limited spatial locality. Consequently, their performance is often constrained by irregular memory accesses rather than computation.

This project will characterize graph applications from the GAP Benchmark Suite, or GAPBS, on a modern high-performance processor model. Because developing a complete processor in an academic setting is infeasible, the project will use Intel Pin for binary instrumentation and the Sniper architectural simulator for development and analysis. Sniper supports Pin-based application instrumentation and can record and replay traces, in a format known as SIFT.

## **Methodology and Objectives**

The first stage will profile and analyze the GAPBS source code. For each selected application, the study will identify the main algorithm function or frequently executed loops. Native profiling tools such as Linux perf will be used to measure execution time, instructions, cycles, IPC, branch misses, and cache misses.

The source code will then be instrumented with a clearly defined Region of Interest, or ROI. The Sniper simulator provides calls that can be used to mark the main graph algorithm or a representative algorithm iteration. Intel Pin will instrument the compiled application, while Sniper's record-trace flow will generate SIFT traces containing instructions and memory references from the marked region. Experiments will use representative GAPBS applications and both synthetic and real-world graphs as inputs.

Once the traces have been created, we will simulate them using the Sniper simulator and model a state-of-the-art processor such as the ones from Intel or AMD. We will characterize the traces/applications and identify the performance bottlenecks they experience on these processors. We will perform a detailed sensitivity analysis to understand these benchmarks by varying different processor and cache structures and assessing their impact on performance. For example, by varying the size of caches and virtual memory structures. Finally, we will analyze the impact of hardware prefetching on performance by integrating a couple of recent hardware prefetchers in Sniper.

## **Expected Outcomes**

The project will deliver modified GAPBS applications with ROI markers, validated Pin/SIFT traces, Sniper-compatible implementation of hardware prefetchers, and a detailed performance and analysis report.

## **Prerequisites**

Familiarity with basic computer architecture and operating systems, especially the fundamentals related to processor pipeline and caches. The project requires strong programming skills in C/C++, Python, and experience in managing a large software system on a Linux-based platform.

## Hardware-Software Co-Design for Edge AI: RISC-V & Spiking Neural Networks on FPGA

**Responsible:** Federico Corradi [f.corradi@tue.nl](mailto:f.corradi@tue.nl)

**Project Description:** Modern edge computing requires highly efficient processing architectures. In this project, students will design and evaluate a custom System-on-Chip (SoC) tailored for neuromorphic workloads. Using the PYNQ-Z2 FPGA, the group will instantiate a soft-core RISC-V processor and modify the system to run Spiking Neural Networks (SNNs). The project emphasizes hardware-software co-design, requiring students to analyze performance bottlenecks and implement hardware acceleration strategies, ranging from custom instruction set extensions to multi-core scaling.

### Weekly Milestones (5 Weeks / 8 hours per week):

- **Weeks 1-2: Baseline System Bring-up:** Instantiate a lightweight open-source RISC-V soft-core (e.g., VexRiscv or PicoRV32) on the FPGA fabric. Compile and execute basic C/Assembly benchmarks to verify memory and peripheral connectivity.
- **Weeks 3-4: Neuromorphic Hardware Acceleration:** Profile a software-based Leaky Integrate-and-Fire (LIF) neuron model on the baseline core. Identify bottlenecks and design a hardware accelerator or custom instruction set extensions (ISA) to optimize spike generation and propagation.
- **Week 5: Evaluation:** Evaluate the final latency, throughput, and hardware resource utilization.

**Supervision & Expectations:** A TA from the NECS group will be available for two 30-minute check-ins per week to guide students through the Vivado/RISC-V toolchain and provide feedback for the final assessment. Please note that this is an advanced, hands-on project that requires autonomous problem-solving to overcome complex engineering challenges.

**Prerequisites:** This project strongly requires prior experience with FPGAs and a solid working knowledge of Vivado, VHDL, and Verilog. Students who do not meet these requirements will face significant difficulties in completing the project successfully.

## Case Study: Designing an Energy-Efficient Multicore Accelerator for LLM Inference

Responsible: Manil Dev Gomony ([m.gomony@tue.nl](mailto:m.gomony@tue.nl))

### Context and objective

Large language models (LLMs) require considerable computation and frequent movement of weights and intermediate data. Their performance and energy efficiency therefore depend not only on the processing cores, but also on the memory hierarchy, interconnect, workload mapping, and scheduling strategy.

In this project, you will act as a hardware-architecture team and design a multicore accelerator for a representative part of LLM inference. Starting from a provided baseline, you will quantitatively explore architectural alternatives using the open-source ZigZag/STREAM framework (<https://github.com/kuleuven-micas/stream>). STREAM models heterogeneous multicore accelerators, estimates per-core costs using ZigZag, and schedules neural-network workloads while accounting for memory and communication.

Your goal is to propose a final architecture that improves latency, energy consumption, or the energy–delay product relative to the baseline, subject to an explicit resource constraint.

### Design challenge

Select a representative LLM building block, preferably a provided SwiGLU feed-forward block, or a small decoder block supplied in ONNX format, and define a realistic inference scenario. For example, consider interactive edge inference for a specified batch size and token-processing rate.

First determine the application requirements and identify the important operations, tensors, and bottlenecks. Next, reproduce and analyze the provided STREAM baseline. Each team member then investigates one complementary part of the design space:

1. **Processing-core design:** number and type of cores, processing-element array dimensions, compute throughput, utilization, and dataflow.
2. **On-chip memory hierarchy:** local and shared memory capacities, memory levels, bandwidth, and the placement and reuse of weights, activations, and intermediate results.
3. **Multicore organization:** homogeneous versus heterogeneous cores, workload partitioning, interconnect bandwidth/topology, and communication overhead.
4. **Mapping and scheduling:** tiling, layer fusion, operator placement, load balancing, and parallel execution across cores.
5. **Data-transfer optimization** (*for a five-person team*): buffering and transfer scheduling that overlap communication with computation, interpreted as prefetching where supported by the model.

A four-person team may combine the final two topics. Responsibilities should be divided, but the experiments and conclusions must form one coherent system-level study.

### Method and expected results

Vary one or more architectural parameters in controlled experiments and compare every design against the same baseline. Report at least:

- execution latency or throughput;

- estimated energy and energy–delay product;
- utilization and, where available, compute-versus-data-movement time;
- memory-capacity and bandwidth requirements;
- the resource cost or complexity of each improvement.

Use plots or Pareto curves to reveal trade-offs rather than presenting only the single best result. Clearly distinguish results produced by the tools from assumptions, extrapolations, and effects that STREAM does not model directly. Finally, combine the most promising individual ideas, evaluate the integrated architecture, and explain why it is preferable for the selected application.

### **Deliverables**

- Reproducible STREAM configuration files and experiment scripts;
- a concise final report describing the application, requirements, baseline, individual explorations, integrated design, results, limitations, and conclusions;
- a final presentation in which each student explains and defends their contribution.

### **Suggested five-week plan**

**Week 1:** Understand the workload and STREAM; define requirements and reproduce the baseline.

**Week 2:** Profile the baseline, identify bottlenecks, and formulate individual hypotheses.

**Week 3:** Explore the assigned design dimensions using controlled experiments.

**Week 4:** combine the strongest ideas and evaluate the integrated architecture.

**Week 5:** Perform sensitivity analysis, document limitations, and prepare the report and presentation.

**Success criterion:** Deliver a quantitatively supported multicore architecture that improves at least one target metric without hiding unacceptable penalties in another metric.